HORIZONX

HorizonX Software Requirements Specification

Frontend Prototype Requirements & Verification Baseline

VERSION 1.0 | 4 November 2026

Document contents

1. Document control
2. Purpose and scope
3. System context and interfaces
4. Functional requirements
5. Non-functional requirements
6. Data requirements
7. Constraints and assumptions
8. Verification status and release caveat
9. Traceability summary

1. Document control

Field	Value
Document	Software Requirements Specification (SRS)
Project	HorizonX (repository/project label)
Current in-app brand	HorizonX references remain; canonical branding is an open decision
Version / date	1.0 / 4 November 2026
System boundary	React/Vite browser application; no backend or database

2. Purpose and scope

This SRS defines the functional and non-functional requirements supported by the current HorizonX frontend prototype. It describes observable behavior and implementation boundaries. It is not a specification for a production exchange, custody system, smart-contract protocol or KYC service.

The system is a browser-based single-page application. It uses local catalog fixtures, browser storage, wallet connectors and an Ethereum Sepolia RPC connection for wallet state/native balance. Most business operations remain simulated or display-only.

3. System context and interfaces

Interface	Description	Current status
Browser	Modern browser runtime for React SPA; session/local storage; history API	Implemented
Wallet connector	Injected provider, MetaMask, WalletConnect and Coinbase Wallet via Wagmi	Implemented; actual behavior depends on installed wallet/provider
Blockchain RPC	Ethereum Sepolia (11155111) for native balance and wallet network state	Implemented
Application API	REST/GraphQL API for accounts, projects, campaigns and analytics	Not implemented
Database	Persistent application storage	Not implemented
Indexer / market data	Token activity, holders, pricing, volume, liquidity and charts	Not implemented; illustrative fixtures used
Smart contracts	Token factory, launch, airdrop, staking and exchange transactions	Not implemented
KYC provider	Identity checks and compliance workflow	Not implemented

4. Functional requirements

ID	Requirement	Priority	Verification
FR-001	The system shall render the home page and shared application shell.	MUST	Open / and verify navigation, page content and shared footer.
FR-002	The system shall expose Explore discovery routes for tokens, projects, presales, launches, airdrops and staking.	MUST	Navigate each route and verify directory content.
FR-003	The system shall expose token dashboard, explorer, My Tokens, create, watchlist, detail, contract and activity routes.	MUST	Open all token routes and verify route resolution.
FR-004	The system shall expose Launchpad routes for presales, fair launches, create flows, detail views and My Launches.	MUST	Open routes and verify forms/listings.

FR-005	The system shall expose Airdrop dashboard, campaign directory, create, detail, claim and My Campaigns views.	MUST	Open routes and verify session/catalog states.
FR-006	The system shall expose Staking dashboard, pool directory, create, detail and My views.	MUST	Open routes and verify available UI states.
FR-007	The system shall expose Exchange/Markets, market token and watchlist views.	MUST	Open routes; verify demo provenance on illustrative market values.
FR-008	The system shall expose Projects, Partners, Account and Docs/Help routes.	MUST	Open routes and verify navigation.
FR-009	The router shall lazy-load page modules and provide an error boundary and not-found route.	MUST	Inspect network chunk loading; navigate unknown path and trigger controlled route error.
FR-010	The system shall allow wallet connection through configured Wagmi connectors.	MUST	Connect/disconnect a test wallet on Sepolia; handle rejection.
FR-011	The system shall expose connected address, connector and supported-network state.	MUST	Verify state changes when account/network changes.
FR-012	The system shall retrieve the connected account's native ETH balance on Sepolia.	MUST	Compare displayed balance with a Sepolia explorer/RPC at the same block/time.
FR-013	The system shall identify unsupported chains and constrain configured active chain support to Sepolia.	MUST	Connect on an unsupported chain and verify warning/switch behavior.
FR-014	The system shall provide a demo wallet mode distinct from a real wallet connection.	MUST	Enter demo mode and verify demo labeling, fixed simulated account and local persistence.
FR-015	The system shall support token creation as a simulation/session receipt only.	MUST	Complete the wizard and verify no contract transaction is submitted.
FR-016	The system shall support session-created presale and fair-launch records where implemented.	SHOULD	Create a record; verify sessionStorage and simulation provenance.
FR-017	The system shall support session-created airdrop campaign records and local Merkle helper behavior where exposed.	SHOULD	Create campaign and inspect local record/proof output; verify no on-chain claim.
FR-018	The system shall distinguish catalog, simulated, session and on-chain provenance.	MUST	Inspect labels/tooltips and verify market fixtures are not marked on-chain.
FR-019	The system shall provide search/filter/sort/pagination controls on applicable directories.	SHOULD	Exercise each available control and verify deterministic result updates.
FR-020	The system shall provide a wildcard not-found page and route error view.	MUST	Request unknown path and trigger an error state.
FR-021	The system shall retain browser-session records only within the storage lifecycle defined by the relevant sessionStorage utilities.	MUST	Refresh within session, open a new session and verify expected record lifecycle.

5. Non-functional requirements

ID	Requirement	Verification

NFR-001	The frontend shall use strict TypeScript compilation settings.	Run npm run typecheck.
NFR-002	The production bundle shall be generated through the Vite build pipeline.	Run npm run build.
NFR-003	ESLint shall be executable against the frontend source.	Run npm run lint and review all findings.
NFR-004	The test command shall execute the intended automated test inventory.	Compare package script with all test files; update and run the command.
NFR-005	The SPA shall support nested-route refresh when the hosting platform is configured with an index.html fallback.	Deploy and refresh a deep link directly.
NFR-006	The UI shall support keyboard navigation, visible focus and semantic labeling for interactive controls.	Manual keyboard and accessibility review.
NFR-007	The UI shall provide usable layouts at mobile, tablet and desktop breakpoints.	Review 375, 430, 768, 1280 and 1440 px.
NFR-008	The UI shall handle wallet rejection, RPC failure, unsupported network and browser-storage failure without false success.	Fault-injection/manual tests.
NFR-009	No secret credential shall be embedded in browser-exposed VITE_ variables.	Inspect environment files and production bundle.
NFR-010	The system shall not represent simulated/catalog data as verified blockchain or live market data.	Review provenance labels, tooltips and copy.
NFR-011	The system shall not claim server-enforced authorization or durable identity records without a backend.	Architecture/security review.
NFR-012	Animations shall respect reduced-motion preferences where supported by the relevant component.	Manual reduced-motion review.

6. Data requirements

Entity/data class	Required fields / semantics	Authority
Token catalog	Stable ID, name, symbol, network label, category, contract string, supply, decimals, status, verification metadata, project relation and market fixture	Static frontend catalog; not chain-verified
Project catalog	Stable ID, name, description, owner display address, status, category, ecosystem, website and token relations	Static frontend catalog
Presale / fair launch	Stable ID, token/project relation, status, schedule/limits where present, amount fields and provenance	Static catalog or session simulation
Airdrop campaign	Stable ID, token relation, status, campaign configuration and optional local Merkle data	Static catalog or session simulation
Staking pool	Stable ID, token relation, status, pool/reward display fields	Static frontend catalog
Wallet session	Address, connector, chain ID, connection status and native Sepolia balance	Wallet provider / RPC; demo mode is simulated
Market record	Price, 24h change, volume, liquidity, market cap and chart series	Simulated fixture; no live provider
Provenance	catalog, simulated, session, onchain	Required to prevent source conflation

7. Constraints and assumptions

- The supported EVM chain is Ethereum Sepolia (11155111) only.
- Catalog records can mention other networks; this does not mean the wallet integration supports them.

- The app is a Vite static SPA and requires an SPA fallback in production hosting.
- Browser storage is not a trusted database and is not shared across devices.
- Wallet connection does not create a platform account or establish legal identity.
- Current market, holder, trade, contract and campaign examples must be treated according to their provenance metadata.

8. Verification status and release caveat

The supplied archive contains 28 TypeScript test files under frontend/src/tests. The npm test script explicitly chains 15 of them, so 13 test files are not invoked by that script. None of the build, typecheck, lint or test commands was executed during this documentation pass. The requirements above define expected verification; they are not a statement that verification has passed.

Production acceptance is blocked until the product owner approves branding and scope, a backend/authentication and data architecture is implemented where required, real data providers and indexers are selected, contract specifications and audits are completed, legal/privacy disclosures are reviewed, and operational monitoring/recovery controls are established.

9. Traceability summary

Requirement group	Primary implementation areas
FR-001 to FR-009, FR-019, FR-020	src/routes.tsx; src/pages; src/components/layout; src/components/market; feature components
FR-010 to FR-014	src/config/wagmi.ts; src/config/chains.ts; src/contexts/wallet-context.tsx; src/components/providers/web3-provider.tsx
FR-015, FR-018	src/components/tokens; src/lib/data-provenance.ts; src/lib/tokens
FR-016, FR-017, FR-021	src/lib/launch-storage.ts; src/lib/airdrop-storage.ts; src/lib/merkle.ts; feature creation components
Data requirements	src/data/catalog; src/data/index.ts; src/types/index.ts
NFR-001 to NFR-004	frontend/package.json; frontend/tsconfig.json; frontend/src/tests
NFR-005	frontend/vite.config.ts; frontend/public/_redirects; hosting rewrite configuration